

A Model for Global Software Development with Cloud Platforms

Luisanna Cocco, Katuscia Mannaro, Giulio Concas
University of Cagliari, Italy
Dipartimento Ingegneria Elettrica ed Elettronica
Email: [luisanna.cocco, mannaro, concas]@diee.unica.it

Abstract—Cloud Computing (CC) is a technological phenomenon that is becoming more and more important. Also Small and Medium Enterprises (SMEs) can increase their competitiveness by taking advantage of CC. This new computing approach promises to provide many advantages, and many SMEs are encouraged to use it. However, CC is still in its early stage – for this reason we think that it is very important to study and assess its impact on SMEs’ management processes.

In this paper we propose a model for studying how Global Software Development can be facilitated using Cloud development environments, compared to a more traditional development environment. We use system dynamics to model and simulate how effective Cloud-based software development environments are for Global Software Development performed by a SMEs. Both studied environments assume a development process based on Scrum agile methodology. The proposed model could be used as a tool by the project managers to understand how Cloud Development Environments might facilitate Global Software Development.

Index Terms—System Dynamics, Modeling, Simulation, Cloud System, Scrum, Global Software Development.

I. INTRODUCTION

In this paper we propose a model for studying the factors related to a development environment on cloud platform – Platform as a Service (PaaS), compared to a development environment set up for an on-premise agile development model. Traditional software applications are based on a model with large upfront licensing costs and annual evergreen support costs. On the contrary, on-demand applications are based on a recurring subscription fee and the cost may increase as the usage of the application increases; in this approach, the possession of the physical location, of the hardware and of the system maintenance is in charge of cloud providers. Specifically, there is a number of issues that need to be considered. We compare two development environment, one on-premise, and the other on-demand, both set up for agile development models, in order to perform Global Software Development (GSD). Among available Agile Methodologies, we choose Scrum due to its popularity.

Scrum was proposed as a possible solution for quickly responding to changing customer requirements, without compromising the quality of the code, and is presently the most used Agile Methodology (AM) [1].

The model proposed is built by using an analysis of feedback loops among the components of the processes, such as requirements, iterations, releases and so on, and through

workflows and delays, to control their dynamics. We simulated the models using a commercial tool available on the market: Vensim¹. Software processes simulation can be useful in various respects – it can improve current processes, or enforce motivation for changes [2].

In order to compare these models, we assume a software project with fixed requirements to be incrementally implemented. To our knowledge, this is the first time that GSD is studied by using system dynamics models.

The remainder of the paper is organized as follows: in Section II we present a brief description of some key software concepts of the two studied software development approaches. Section III gives an overview of the software process simulation techniques and provides an introduction of the simulation model used in this study and developed in section V. Section IV presents related researches, while section V describes the details of the simulation model. Section VI shows the analysis of data obtained from the simulation of the two models described in Section V. Section VII summarizes the conclusions of our research, and ends the study with recommendations for future works.

II. AN OVERVIEW OF SCRUM PROCESS, AND OF GLOBAL SOFTWARE DEVELOPMENT.

In this section we take a look at Scrum, the software development approach considered in our work. Agile Methodologies, so named in 2001 in the Agile Manifesto [3], have been introduced in response to “heavyweight” software development methodologies, in which all process phases (planning, design, development, testing and deployment) are performed in a sequential series of steps. Among them, Scrum is an Agile process tool [4] based on incremental development. It uses pull scheduling and emphasizes the need to deliver releasable software often.

The original term “Scrum” comes from a study by Takeuchi and Nonaka [5] that was published in 1986 in the Harvard Business Review. In 1993 Jeff Sutherland developed the Scrum process at Easel Corporation, by using their study and their analogy, as well as the name of the process as a whole. Finally, Ken Schwaber [6], [7] formalized the process for the worldwide software industry in the first published paper on Scrum at OOPSLA 1995. Scrum [8] is a simple agile

¹www.vensim.com

framework, adaptable also to different contexts into the software development. [9]. Scrum has three roles (Product Owner, Scrum Master, Team), three ceremonies (Sprint Planning, Sprint Review, and Daily Scrum Meeting), and three artifacts (Product Backlog, Sprint Backlog, and Burndown Chart). Adopting Scrum implies to use time-boxed iterations and to break the work into a list of smaller deliverables, ordered according to a priority given by the Product Owner. Changes to requirements are not accepted during the iteration, but are welcomed otherwise. Scrum projects are organized with the help of daily Scrums – 15 minutes update meetings – and monthly Sprints, or iterations, that are designed to keep the project flowing quickly. Generally, at the end of every iteration the team releases working code, and a retrospective meeting is held, also to look for ways to improve the process for the next iteration. Finally, in the case of large projects, the use of a "Scrum of Scrums" meeting is expected. This approach is used to coordinate the work of multiple Scrum teams. One person for each team attends Scrum of Scrum meetings. These meetings are analogous to the Daily Scrum Meeting, and usually happen two or three times a week.

A common definition of Global Software Development is a software development process at geographically separated locations. For this reason, GSD involves communication for information exchange, coordination of teams, activities and artifacts so they contribute to the overall objective, and finally control of teams. Many challenges meet into GSD [25], [26] – these are geographic, cultural, linguistic and temporal. The distance and the lack of overlapping working hours create a negative impact on software projects, create problems in the knowledge transfer, and as a consequence communications gaps or ambiguity on technical aspects may occur. Cultural diversities may bring to an unequal distribution of work, lack of trust and fear, from which cost increases, poor skill management and reporting issues may arise. Linguistics and temporal diversities can instead lead to issues in knowledge transfer, communication and project visibility.

The GSD can be facilitated using the Cloud. CC is a delivery model for software, platforms and infrastructures. Cloud providers have got the possession of physical location, hardware, and system maintenance. Enterprise users access cloud services via Internet from anywhere and at any time. Users usually pay a subscription fee, and can run on a robust infrastructure, a single instance of system. Cloud services are in fact delivered from a "multi-tenant" system; there is a single instance of software running, but many individual or enterprise customers use this system concurrently for their own necessities.

Cloud Computing contains 3 different paradigms ([27]):

- Software as a Service (SaaS),
- Platform as a Service (PaaS),
- Infrastructure as a Service (IaaS).

The first is a cloud hosting model where the providers offer a complete end-user application. The second is a model that provides a full or partial application development environment. Providers offer in addition to the hardware, a platform in-

cluding OS, middleware and runtime environment. Finally the last, Infrastructure as a Service is a model that provides a full computer infrastructure, server, router, storage, hardware and virtualization software. Moreover these cloud deployment models can be Private, Public or Hybrid depending on the scope of the cloud solutions. In the public model the cloud computing services are accessible to a large number of end-users, on the contrary in the private model the companies realize a cloud computing environment that allows to store the data within their operational structure with obvious advantages in terms of security and privacy. In this last case the cloud services are accessible only by authorized end-users. The third model is a system that is in the middle between the two models described above. It allows to enjoy the benefits of public model especially in terms of costs and scalability, and at the same time to have the guaranteed standards of management and security typical of the private models.

GSD seems to be perfectly integrable with CC. In fact, GSD can be potentially improved thanks to the main characteristics of Cloud Computing: virtualization, reduced cost, scalability, infrastructure, performance and Multi Tenancy Support [26]:

- *Virtualization*: it allows to enhance the infrastructure to satisfy growing demand for services;
- *reduced cost*: Cloud services are based on the "pay as you go" model, so the subscribers pay only for the resources they use;
- *scalability*: it allows dealing successfully peak demand and avoid under-utilization of computer resources;
- *infrastructure*: the underlying infrastructure is invisible to the customers, who do not have to worry about it;
- *performance*: each consumer can set up on the Cloud the level of performances more suited to her needs;
- *multi tenancy support*: Cloud services are delivered from a multi-tenant system; a single instance of the system is running, but many consumers can use the system, as their workloads are isolated to ensure security and privacy of their data.

The positive implications that Cloud Computing can provide, with respect to traditional software development environments are many. Among the benefits linked to Cloud services, ([21], [22], [23], and [24]), we can name:

- no need to invest upfront in infrastructure;
- no hardware or software required for cloud services;
- no maintenance of the infrastructure that runs development applications;
- optimum utilization of in-house IT resources;
- building only what is needed, eliminating anything that does not add value;
- faster time to market;
- pay per use;
- easy setup and deployment of applications, uploading them on the Cloud;
- easy integration with other enterprise solutions;
- efficient collaboration among teams during all software development phases;

- access to a global environment;
- highly customized environment;
- costs savings.

The model proposed in Section V helps to highlight these issues, offering a way to estimate the potential benefits. The simulator developed in Vensim environment is a simple, customizable tool that each enterprise can set up in order to study how these benefits can influence its development process.

III. SOFTWARE PROCESS SIMULATION

Simulation is a research methodology that allows to estimate the behavior of a system under some conditions, and can be applied to different research strands. In the simulation domain there are three main techniques: system dynamics, agent-based simulation and discrete event simulation.

In [11], Kellner, Madachy and Raffo discussed the “why, what and how” of software process simulation. We agree with them that there are several advantages of simulating models of software processes. Simulation models can be used for planning, control and operational management, process improvement and technology adoption, training and learning.

The System Dynamics approach (SD) for modeling and simulating, introduced by Jay W. Forrester [12] of the Massachusetts Institute of Technology during the mid-1950s, is suitable to analyze and model non-linear and complex systems containing dynamic variables that change over time.

SD allows to capture accurately the effects of feedback and to represent clearly the relationships between dynamic variables, unlike the discrete event simulation that instead allows to accurately represent sequential activities, and entities or attributes. SD is suitable to analyze and model non-linear and complex systems containing dynamic variables that change over time. The system under study is described through a diagram whose variables are divided into stocks, flows and auxiliary variables and connect among them through links. The stock variables represent values increasing or decreasing through time and depend on their previous values, flow variables are linked directly to stocks and determining their change, finally the auxiliary variables are constants of the system. In SD the analysis is carried out by identifying cause-effect relationships inside the system. Positive and negative feedback loops are represented by a causal loop diagram. In this work we used the commercial tool called Vensim, that allows to construct a model and to examine the structure using structural analysis tools, so to simulate the model moving around model parameters to see how it responds, and examine interesting behaviour in more detail using graphs and tables.

IV. RELATED WORK

System dynamics modeling has been used in similar research on software development process, where there are multiple and interacting software processes, time delays, and other non-linear effects such as communication level, amount of overtime and workload, schedule pressure, budget pressure, rate of requirement change, and so on.

In the field of the Agile Methodologies, many system dynamics models were introduced. The main goals of these researches aim to better understand the agile process and to evaluate its effectiveness. Most of the performed research was made on Extreme Programming (XP), or generic AMs. Other processes such as Scrum, however, are almost absent. For example, Chichacly in [14] investigated when AMs may work by using System Dynamics modeling, and comparing AMs with a traditional waterfall process. In [15] the author explored whether agile project management had a unique structure, or would fit within the generic conceptually formed system dynamic project management structures. An analysis of factors that impacts on productivity during agile web development and maintenance phases was conducted by Xiaoying Kong et al. [16]. Another analysis published in [17] gives both theoretical insights into the dynamics of agile software development, and practical suggestions for managing these projects.

Two works closely linked to ours, but not concerning system dynamics modeling, are [25] and [26]. In [25] Hossain, Babar, Paik, and Verner discuss the use of Scrum practices in GSD projects, and identify key challenges, due to global project distribution that restricts the use of Scrum. In [26] instead, the authors present the challenges encountered in globally dispersed software projects and propose to exploit Cloud Computing characteristics and privileges both as a product and as a process to improve GSD. In a more and more globalized world the relationship between culture and management of remote work is an unavoidable issue to face, so they proposed to exploit Cloud Computing both as a product and as a process to manage the many challenges in terms of culture, management, outsourcing, organization, coordination, collaboration, communication, development team, development process and tool. The authors identified GSD challenges and requirements that a cloud architecture could solve. For example, they affirm that cloud services, as a product, ensure interactions among different activities, while as a process it could allow resource sharing, infrastructure and application resources, but also software resources and business processes. Again the issue of geographically distance could be solved with Platform as a Service. It does not require any kind of downloads and installations: therefore used as a product it supports geographically distributed teams, instead when used as a process, it can help to overcome many limitations encountered in software evolution, reuse and deployment. The work which inspired our model is [21]: a practical experience in the application of some agile software development practices, as Scrum model, to Azure application development. Azure Services Platform is an application platform on the cloud and it offers PaaS capabilities, which allow application to be built and consumed from both on-premise and on-demand environments. This paper starts from several questions about the interactions between cloud computing and agile software development and attempts to discuss their potential advantages. In fact the authors show how setting up a development environment on the Azure platform helps enhance the agile practices.

Beginning from these works, and in particular this last one

just mentioned ([21]), we realized a simple model to study how effective Cloud-based software development environments are for Global Software Development to facilitate the daily work.

V. MODEL STRUCTURE

According to SD modeling, and as reported in [18], our model is represented in terms of stocks and flows in an out of them. The first step has been to identify what are the flows in the main process, which in our case are the project requirements. The auxiliary variables control the “valves” acting on the alternative outflows of each stage.

We have described the process subdividing the work into a set of Sprint backlogs, each including a random number (in the range 15-21) of features selected from a Gaussian distribution. These backlogs are developed during short fixed-length iterations of 2 weeks.

The *requirements development rate* is the speed at which requirements flow to “Fraction Work Done”. This rate is determined by the team’s productivity, by the number of developers and by the error rate and it is defined by the following equation (expressed in Vensim language):

We set an average productivity of 0.025 requirements per hour (the number of requirements developed in one hour) because we assume that a developer is able to implement a requirement in about 5 days. Moreover, the fewer errors there are in the process, the sooner the project will be finished.

Fig. 1. Simplified version of the proposed model.

Parameters	On-Demand / On-Premise
n° Features in Scrum	Integer[Random Normal (15 , 21)]
Scrum for Planning Phase	2 hours
meeting of sprint planning	Integer[Random Normal (4 , 8)]
retrospective meeting	1 hours
daily Scrum meeting	1 hours
Bug introduced during the development	Random Normal (0.0071, 0.0095)
Problem in the software design	Random Normal [0.001, 0.0014]
Effect of uncertain customer requirements	Random Normal (0.0047, 0.0071)

As requirements are implemented, they flow into the "Integration Testing", "System Testing" and "User Acceptance Testing" stock. If the tests are successfully passed, then the requirements are accepted and are considered completed (the accepted requirements flow into the "Production Environment" stock). Otherwise, a rework must be performed, which includes a delay due to the correction. From "Production Environment" stock the requirements flow into the "Live" stock, and the work is finished.

In the following Table I we report some initial parameters we used, according to [18]. Note that these values have been gathered through a review of the literature.

A. Factors affecting software development.

These effects influence project outcomes and determine the system development speed. In our model, an important role is assigned to delays. In traditional development environments, at the beginning of a project development lifecycle, considerable time and effort are required for procuring

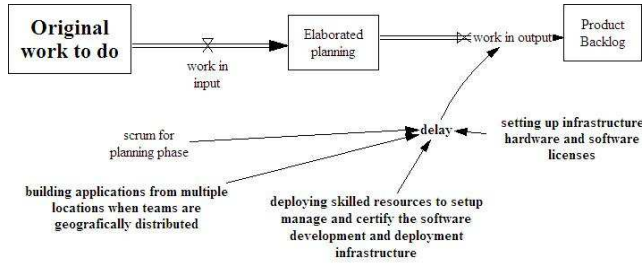


Fig. 2. Delays in the planning.

hardware and software licenses, and during the development additional time might be spent for upgrading this environment. On the contrary, in Cloud development environments, the infrastructure is readily available, and system maintenance and system updates of the cloud server will be taken care of by the cloud providers, and not by the developers. Moreover, Production and Testing Cloud environments are accessible anytime, anywhere: any team member and user can work and build applications referring just to one location, with no need to coordinate multiple locations. In this way, significant time and effort will be saved, and users will use all their resources for creating value for their business.

In our model, time and effort spent at the beginning of project development lifecycle are modeled by the following variables: *setting up infrastructure hardware and software licenses*, *deploying skilled resources to setup, manage and certify the software development and deployment infrastructure*, *building applications from multiple locations when teams geographically distributed* are added. These variables, some of which were taken from the work of Dumbre et al. [21] lead to delays. Another variable should be added to these ones, for planning and creating the Backlog. This variable is indicated as *scrum for planning phase*. These four variables are reported in Fig. 2.

Time and effort for the maintenance of traditional environments influence the project during the development phase. This may happen during the Scrum sprint, and is modeled with the variable called *maintenance during the course of the development and for additional overhead of handling software and hardware patching upgrades*, shown in Fig. 3.

Regarding what affects the length of Scrum iteration, Scrum prescribes roles such as the Product Owner – a single person representing the customer's interest who is in charge of backlog prioritization, and who shall answer to questions about requirements. Moreover, a Sprint Planning Meeting, a Sprint Retrospective Meeting and Scrum of Scrums meeting are prescribed, respectively to plan the iteration, at the end of every sprint, and to coordinate more teams that work at geographically separated locations. The sprint review meeting aims to discuss what went well and what to improve in the

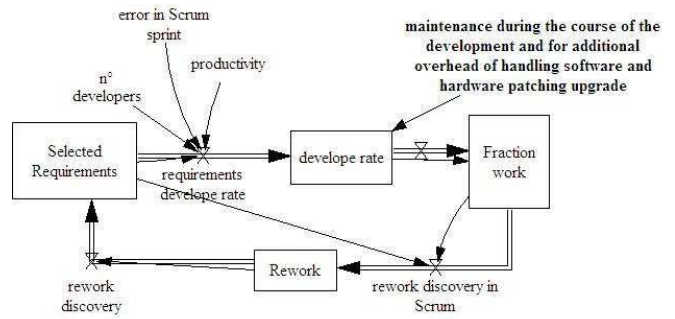


Fig. 3. Development phase.

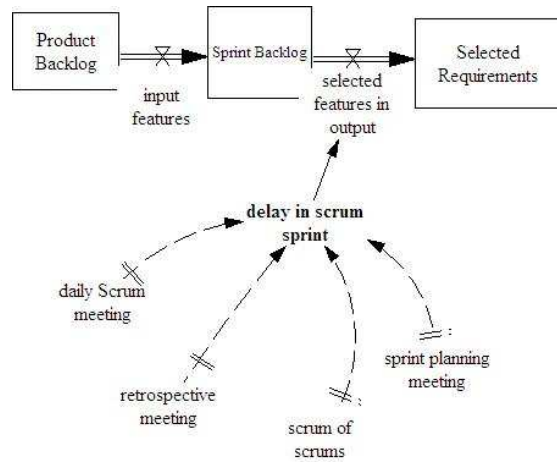


Fig. 4. Delay for meetings.

next sprint. In our model, these meetings and the activity of the Product Owner are modeled through the following variables: *Sprint Planning Meeting*, *Retrospective Meeting*, *Scrum delay* and *Scrum of Scrums* illustrated in Fig. 4.

As regards the effects of errors, Fig. 5 shows the causal loop effect on the error rate. We identify a reinforcing feedback loop (positive feedback) where cause and effect can be circular and an effect strengthens its own cause, or where sometimes a cause has multiple opposing effects. For example the *effect of uncertain customer requirements* reinforces *error rate*.

As mentioned before, a fraction of the work is considered to be done incorrectly due to three types of error: *effect of uncertain customer requirements*, *problem in the software design*, *bug introduced during the development*. Only the *bug introduced during the development* error passes through the *rework discovery in Scrum* valve characterized by the following equation:

$$\text{rework discovery in Scrum} = \text{IF THEN ELSE}(\text{Selected Requirements} < 0.05, \text{Fraction work done} \times (\text{error at the end$$

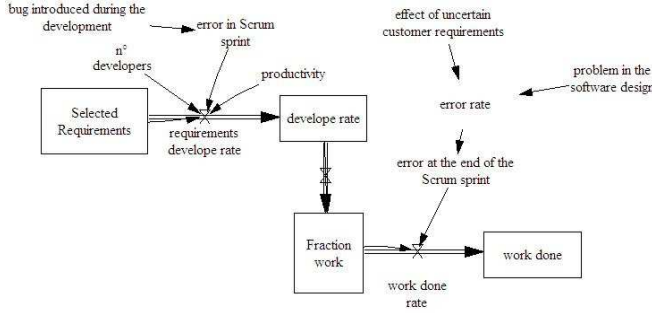


Fig. 5. Causal loop diagram of error rate.

of the Scrum sprint), 0)

The two other errors can be discovered only at the end of the iteration.

Finally, we analyze the time and effort spent in the different test environments, and in the production environment. Under traditional development environment, much time and effort are spent to write verbose installation scripts or release notes, and for the setup of system testing, integration testing and user acceptance testing, with the aim to obtain a product released under rigorous test and validation. Moreover, the developers must often deliver rapid prototyping and demos to obtain immediate feedback from the end user. This may become a real challenge if the user and the developers work in different locations.

In cloud development environments, instead, these steps are simpler than in a traditional environment. The deployment process is simplified, there is no need of any separate packaging efforts; to pass from development environment to testing environment, and from here to production environment does not require any additional step. Prototypes and demos can be made accessible immediately to customers for eliciting feedback in a short time. Code can pass from one environment to another without writing deployment script to set up the application in the respective environments. What has just been said is modeled by two variables: *creating and managing different test environments* and *creating and managing production environment prototyping and demos* introduced when the requirements are moved from *work done to test* to a different testing environment, and then are deployed to *production environment*. The latter variables are shown in Fig. 6.

All the variables described above represent time and effort spent in the development process, leading to process delays, which mean further overheads of cost and effort.

VI. RESULTS

The system dynamic simulation model described in the previous sections has been proposed for analyzing how the Cloud approach can facilitate the GSD with respect to the traditional approach. We studied the two different approaches, and for switching between them, we introduced in the model

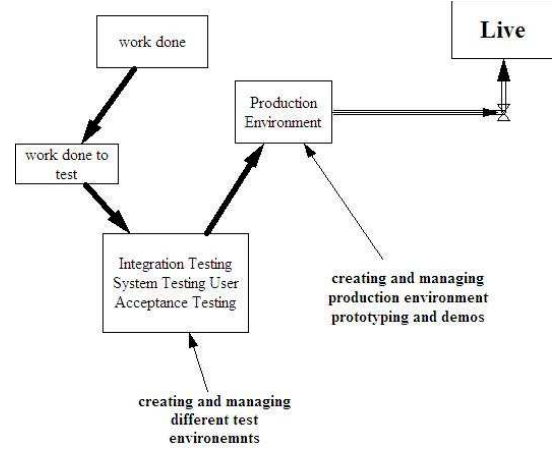


Fig. 6. Delay for testing and production environments.

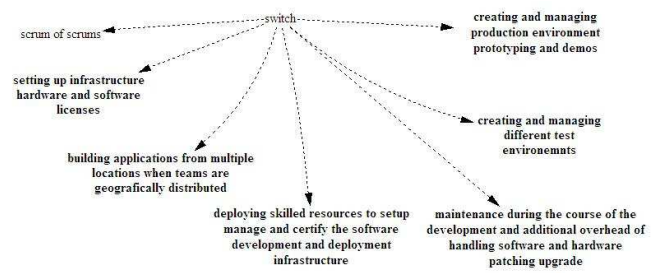


Fig. 7. Work done.

a variable called *switch*. If *switch* value is equal to 0, the modeled environment is the Cloud, if *switch* is equal to 1 the environment is traditional. In figure 7 we report the variable *switch* and all the variables that assume different values in the two type of systems studied.

The results have been obtained setting up the initial variables according to the considerations described in previous Sections and it is just an example showing how our simulator works, and what can be found by using it.

In figure 8 we report the time trend of the work done, and we can notice how the development rate of software is higher in Cloud system than Traditional system.

Our model was simulated implementing only the variables mentioned in Section V-A, because we chose an hypothetical company with special needs, but it is important to note that several aspects can be taken into account to model companies with different needs, and so the conclusions could be different than those obtained by us.

VII. CONCLUSIONS

The system dynamic simulation model described in the previous sections has been proposed for highlighting how a Global Software Development environment on the Cloud Platform may facilitate Global Software Development with

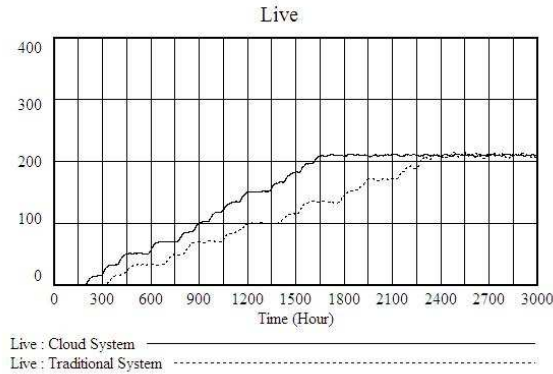


Fig. 8. Live.

respect to an environment set up on-premise, using an agile development method. Our aim is to propose a simple tool that every small or medium enterprise can customize and use, in order to verify if a development environment for Global Software on Cloud System could reduce the costs and the time compared to a development environment set up on-premise. Therefore, the results reported in this work are just a simple example to show the potentialities of our model.

We developed a simplified system dynamics model to describe the behavior of two different approaches to the Global Software Development under similar starting conditions. The development environments analyzed are very complex, and for this reason, in order to avoid a too complex model, we included in our simulator only the variables that in our opinion influence mostly the processes. So, our study has been carried out under some limiting assumptions that could threaten its validity. The proposed model needs to be further elaborated and validated, for example by adding new variables or new relationships among factors. However, this is a first model and one of our future aims is to better answer some questions applying it into detailed case study. This will be the subject of our future work, which will include studies to empirically validate the simulation results using data from GSD real projects and carried on using both Cloud and Traditional environments. Unfortunately it is very difficult to find enterprises available to give answers about any questions related to their software process practices (in particular if you ask about their efficiency, productivity, and so on). This is also one of the reasons which we preferred to start our analysis by implementing a specific model whose parameter values were taken from the literature.

REFERENCES

- [1] VersionOne: State of Agile Survey 2009. Available on the web: http://www.versionone.com/pdf/2009_State_of_Agile_Development_Survey_Results.pdf, Access Date: November 2010.
- [2] Carina Andersson, Lena Karlsson, Josef Nedstam, Martin Höst, Bertil I Nilsson: Understanding Software Processes through System Dynamics Simulation: A Case Study. In: ECBS, Ninth Annual IEEE International Conference and Workshop on the Engineering of Computer-Based Systems, pp. 0041, (2002).
- [3] URL: <http://agilemanifesto.org/>
- [4] Henrik Kniberg and Mattias Skarin. Kanban and Scrum making the most of both. Managing Editor: Diana Plesa. Enterprise software development series. InfoQ. USA. ISBN 978-0-557-13832-6 (2010).
- [5] Takeuchi H. and Nonaka I.: The New New Product Development Game, Harvard Business Review, January-February (1986).
- [6] Schwaber Ken: Scrum Development Process, White Paper, (1997).
- [7] Schwaber Ken: Agile Project Management with Scrum, Microsoft Press, Redmond, WA, (2004).
- [8] Scrum Alliance, <http://www.scrumalliance.org>
- [9] Marchesi M., Mannaro K., Uras S., Locci M.: Distributed Scrum in a Research Project Management Agile Processes. In Software Engineering and Extreme Programming, Lecture Notes in Computer Science, Volume 4536/2007, pp. 240-244, (2007).
- [10] James P. Womack, Daniel T. Jones, Daniel Roos: The Machine That Changed the World : The Story of Lean Production. Harper Perennial, (November 1991).
- [11] Marc I. Kellner and Raymond J. Madachy and David M. Raffo: Software process simulation modeling: Why? What, How?. Journal of Systems and Software, vol. 46, pages 91-105, (1999).
- [12] J.W. Forrester: Industrial dynamics. Cambridge, MA: MIT Press, (1961).
- [13] R. H. Martin and David M. Raffo: A Model of the Software Development Process Using Both Continuous and Discrete Models. International Journal of Software Process Improvement and Practice, vol. 5, number 2-3, pages 147-157, (2000).
- [14] Chichakly Karim: Modeling Agile Development: When is it Effective?. Proceedings of the 2007 System Dynamics Conference. Boston, MA. Print.
- [15] Warren W. Tignor : Agile Project Management. International Conference of the System Dynamics Society, Albuquerque, NM 2009, July 26 -July 30, 2009.
- [16] Kong Xiaoying, Liu Li, Lowe David: Modelling an Agile Web Maintenance Process. (2005).
- [17] Kim E. van Oorschot, Kishore Sengputa, Luk N. van Wassenhove: Dynamics of Agile Software Development. Proceedings of the 27th International Conference of the System Dynamics Society, July 26-30, (2009) Albuquerque, New Mexico, USA.
- [18] Luisanna Cocco, Katuscia Mannaro, Giulio Concas, and Michele Marchesi: Simulating Kanban and Scrum vs Waterfall with System Dynamics, XP2011
- [19] P. Wernick and T. Hall: The impact of using pair programming on system evolution a simulation-based study. In: 20th IEEE International Conference on Software Maintenance, Proceedings, pp. 422-426 (2004).
- [20] Conte S. D., Dunsmore H. E., Shen V.Y.: Software Engineering Metrics and Models. Benjamin/Cummings, (1986).
- [21] Amit Dumbre, Satha Priya Senthil, Sidharth Subhash Ghag: Practising Agile software development on the Window Azure platform, White Paper, May 2011.
- [22] Salesforce: Agile Development Meets Cloud Computing for Extraordinary Results at Salesforce.com, White Paper, 2008.
- [23] Collabnet: Reinforcing Agile Software Development in the Cloud. Why the Cloud is Advantageous for Agile, and for Accelerating its Enterprise-wide Adoption, White Paper, 2011.
- [24] Yash Talreja: Lean Agile Methodologies Accentuate Benefits of Cloud Computing, 2010.
- [25] Emam Hossain, Muhammad Ali Babar, Hye-young Paik, June Verner: Risk Identification and Mitigation Processes for Using Scrum in Global Software Development: A conceptual Framework, 2009 IEEE.
- [26] Sajid Ibrahim Hashmi, Victor Clerc, Maryam Razavian, Christina Manteli, Damiani Andrew Tamburri, Patricia Lago, Elisabetta Di Nitto, Ita Richardson: Using Cloud to Facilitate Global Software Development Challenges, 2011 IEEE.
- [27] Raffaele Giordanelli, Carlo Mastroianni, The Cloud Computing Paradigm: Characteristic, Opportunities and Research Issues, RT-ICAR-CS Aprile 2010.